

Advanced Reading in Computer Vision (MAT3563)

LAB 02.2: PRETRAINED MORDEN CNN – TRANSFER LEARNING

A. SỬ DỤNG THƯ VIỆN CHO MỘT SỐ MÔ HÌNH CNN

Trước hết, để thực nghiệm phần này chúng ta cần đọc thêm về bài giảng (slide) một số mô hình CNN hiện đại có đính kèm và khái niệm Pre-trained.

Dưới đây chúng ta sử dụng thư viện TensorFlow.Keras để gọi một số mô hình CNN hiện đại. Danh sách các mô hình có sẵn trong thư viện này cùng với tham số cụ thể như trong bảng sau:

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7
NASNetMobile	23	74.4%	91.9%	5.3M	389	27.0	6.7
NASNetLarge	343	82.5%	96.0%	88.9M	533	344.5	20.0
EfficientNetB0	29	77.1%	93.3%	5.3M	132	46.0	4.9
EfficientNetB1	31	79.1%	94.4%	7.9M	186	60.2	5.6
EfficientNetB2	36	80.1%	94.9%	9.2M	186	80.8	6.5
EfficientNetB3	48	81.6%	95.7%	12.3M	210	140.0	8.8
EfficientNetB4	75	82.9%	96.4%	19.5M	258	308.3	15.1
EfficientNetB5	118	83.6%	96.7%	30.6M	312	579.2	25.3
EfficientNetB6	166	84.0%	96.8%	43.3M	360	958.1	40.4
EfficientNetB7	256	84.3%	97.0%	66.7M	438	1578.9	61.6
EfficientNetV2B0	29	78.7%	94.3%	7.2M	-	-	-
EfficientNetV2B1	34	79.8%	95.0%	8.2M	-	-	-
EfficientNetV2B2	42	80.5%	95.1%	10.2M	-	-	-
EfficientNetV2B3	59	82.0%	95.8%	14.5M	-	-	-
EfficientNetV2S	88	83.9%	96.7%	21.6M	-	-	-
EfficientNetV2M	220	85.3%	97.4%	54.4M	-	-	-
EfficientNetV2L	479	85.7%	97.5%	119.0M	-	-	-
ConvNeXtTiny	109.42	81.3%	-	28.6M	-	-	-
ConvNeXtSmall	192.29	82.3%	-	50.2M	-	-	-
ConvNeXtBase	338.58	85.3%	-	88.5M	-	-	-
ConvNeXtLarge	755.07	86.3%	-	197.7M	-	-	-
ConvNeXtXLarge	1310	86.7%	-	350.1M	-	-	-

Chú ý:

- Tất cả các mô hình nói trên đều đã được Pre-Trained trên tập ImageNet (~ 1.2 triệu mẫu, 1000 classes), với tham số phần weights mặc định là "imagenet".
- Nếu chúng ta download bộ trọng số này về máy thì sửa nó thành đường dẫn đến tệp trọng số.
- Nếu chúng ta muốn train lại, đặt tham số weights thành **None**

Dưới đây chúng ta nêu một số ví dụ về các gọi/sử dụng các mô hình nói trên trong lớp bài toán (công việc) dự đoán phân loại một hình ảnh.

A.1. PRETRAINED WEIGHTS FOR PREDICTION

1. VGG 16 - 19

Hàm dựng lớp mô hình như sau:

```
keras.applications.VGG19(  
    include_top=True,  
    weights="imagenet",  
    input_tensor=None,  
    input_shape=None,  
    pooling=None,  
    classes=1000,  
    classifier_activation="softmax",  
    name="vgg19",  
)
```

- include_top thiết lập việc có hay không 3 layers fully connected trên cùng. Nếu đặt False đồng nghĩa với việc chúng ta chỉ lấy phần layers tích chập ở phía trước để trích xuất đặc trưng
- weights: Như đã nói ở trên
- classes: số lượng tùy chọn các lớp để phân loại hình ảnh, chỉ được đặt lại nếu include_top = True và nếu weights = None (để trống).
- classifier_activation: string hoặc callable. Hàm kích hoạt để sử dụng trên lớp "top". Bị bỏ qua trừ khi include_top=True. Đặt classifier_activation=None để trả về logits của lớp "top". Nếu weights là pre-trained, classifier_activation chỉ có thể là None hoặc "softmax".
- input_shape: tùy chọn, chỉ được thiết lập nếu include_top = False (nếu không, định dạng đầu vào phải là (224, 224, 3) (với định dạng dữ liệu channels_last) hoặc (3, 224, 224) (với định dạng dữ liệu "channels_first"). Nó phải có chính xác 3 kênh đầu vào và chiều rộng và chiều cao không được nhỏ hơn 32

Ví dụ về việc sử dụng:

```
from keras.applications.vgg19 import VGG19  
from keras.applications.vgg19 import preprocess_input  
from keras.models import Model  
import numpy as np  
  
base_model = VGG19(weights='imagenet')  
model = Model(inputs=base_model.input, outputs=base_model.get_layer('block4_pool').output)  
  
img_path = 'elephant.jpg'  
img = keras.utils.load_img(img_path, target_size=(224, 224))  
x = keras.utils.img_to_array(img)  
x = np.expand_dims(x, axis=0)  
x = preprocess_input(x)  
  
block4_pool_features = model.predict(x)
```

2. RESNET50

Mẫu hàm dựng của lớp

```
keras.applications.ResNet50(  
    include_top=True,  
    weights="imagenet",
```

```

input_tensor=None,
input_shape=None,
pooling=None,
classes=1000,
classifier_activation="softmax",
name="resnet50",
)

```

Các tham số xác định tương tự như VGG16-19. Ví dụ minh họa việc sử dụng cho bài toán phân loại ảnh:

```

import keras
from keras.applications.resnet50 import ResNet50
from keras.applications.resnet50 import preprocess_input, decode_predictions
import numpy as np

model = ResNet50(weights='imagenet')

img_path = 'elephant.jpg'
img = keras.utils.load_img(img_path, target_size=(224, 224))
x = keras.utils.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

preds = model.predict(x)
# decode the results into a list of tuples (class, description, probability)
# (one such list for each sample in the batch)
print('Predicted:', decode_predictions(preds, top=3)[0])

```

- Các mô hình Inception/Xception/EfficientNet được khai báo và các tham số hoàn toàn tương tự. Tên các Class ứng với các mô hình là:
 - from keras.applications.Xception import Xception – đầu vào mặc định 299x299.
 - from keras.applications.inception_v3 import InceptionV3 – đầu vào mặc định 299x299.
- Các bộ trọng số có thể được cung cấp từ giáo viên hoặc để mô hình tự động tải về.

Bài tập thực hành:

- Sử dụng các mô hình nói trên, thực hiện phân loại tập ảnh Chó – Mèo – Gấu trúc (Dogs – Cats – Pandas), với tập Validation là 500.
- So sánh các mô hình dựa trên các tiêu chí:
 - Độ chính xác
 - Thời gian load mô hình
 - Thời gian predict trung bình
- So sánh độ chính xác với mô hình CNN tự xây dựng trước kia.

A.2. TRANSFER LEARNING & FINE – TUNING MODEL (TENSORFLOW BASED)

Transfer learning bao gồm việc lấy các đặc điểm đã học được trên một vấn đề (hoặc lĩnh vực dữ liệu) và tận dụng chúng trên một vấn đề mới (miền dữ liệu khác), tương tự. Học chuyển giao thường được thực hiện cho các công việc mà tập dữ liệu của chúng ta có quá ít dữ liệu để đào tạo mô hình toàn diện từ đầu.

Xét riêng trong ngữ cảnh các phương pháp học sâu (DeepLearning) – Một minh họa phổ biến về Transfer Learning là quy trình làm việc sau:

1. Lấy các lớp từ mô hình đã được đào tạo trước đó.
2. Đóng băng chúng để tránh phá hủy bất kỳ thông tin nào chúng chứa (bao gồm các trọng số đã học được) trong các vòng đào tạo sau này.
3. Thêm một số lớp mới có thể đào tạo được lên trên các lớp đã đóng băng. Các lớp mới sẽ học cách biến các đặc trưng cũ thành dự đoán trên một tập dữ liệu mới.
4. Đào tạo các lớp mới trên tập dữ liệu của chúng ta hiện có.

Bước cuối cùng, tùy chọn, là **FINE TUNING**, bao gồm việc giải nén toàn bộ mô hình bạn đã thu được ở trên (hoặc một phần của nó) và đào tạo lại nó trên dữ liệu mới với tốc độ học rất thấp. Điều này có khả năng đạt được những cải tiến có ý nghĩa, bằng cách điều chỉnh dần dần các tính năng được đào tạo trước cho dữ liệu mới.

Đóng băng các lớp: hiểu thuộc tính **trainable** (Trọng số có thể đào tạo)

Layers và Models có ba dạng trọng số nếu xét trên thuộc tính huấn luyện được hay không:

- **Weights** là danh sách tất cả các biến trọng số của lớp.
- **trainable_weights** là danh sách những dữ liệu cần được cập nhật (thông qua phương pháp giảm dần độ dốc) để cực tiểu hóa hàm tổn thất trong quá trình huấn luyện (training).
- **non_trainable_weights** là danh sách những thứ không được đào tạo. Thông thường, chúng được mô hình cập nhật hoặc sử dụng để tính toán trong quá trình tính toán forward.

Ví dụ: layer Dense có 2 trọng số loại có thể đào tạo (kernel hay weights & bias)

```
layer = keras.layers.Dense(3)
layer.build((None, 4)) # Create the weights

print("weights:", len(layer.weights))
print("trainable_weights:", len(layer.trainable_weights))
print("non_trainable_weights:", len(layer.non_trainable_weights))
```

OUTPUT:

```
weights: 2
trainable_weights: 2
non_trainable_weights: 0
```

Hầu như tất cả các trọng số của layers convolution/dense (fully connected) đều là trọng số có thể đào tạo. Layer tích hợp duy nhất với trọng số không thể đào tạo là lớp **BatchNormalization**. Nó sử dụng các trọng số không thể đào tạo là tham số của phân bố xác suất ứng với mẫu, ví dụ giá trị kỳ vọng và phương sai của chính dữ liệu đầu vào trong quá trình đào tạo.

Ví dụ: Layer BatchNormalization có 2 trọng số có thể huấn luyện và 2 trọng số không thể huấn luyện

```
layer = keras.layers.BatchNormalization()
layer.build((None, 4)) # Create the weights

print("weights:", len(layer.weights))
print("trainable_weights:", len(layer.trainable_weights))
print("non_trainable_weights:", len(layer.non_trainable_weights))
```

OUTPUT:

```
weights: 4
trainable_weights: 2
non_trainable_weights: 2
```

Ngoại trừ Layer **BatchNormalization**, các Layers khác và cả Models đều có thuộc tính **trainable** – Kiểu **boolean (False/True)**, cho phép thiết lập tham số trong Layer đó có thể đào tạo hay không. Giá trị của thuộc tính được thiết lập mặc định là True và có thể thay đổi:

- Đặt **<layer | model>.trainable = False** sẽ đổi tất cả trọng số của layer từ có thể đào tạo sang không thể đào tạo. Điều này được gọi là "đóng băng" layer: trạng thái của một layer đóng băng sẽ không được cập nhật trong quá trình đào tạo (cả khi đào tạo với **fit()** hoặc khi đào tạo với bất kỳ vòng lặp tùy chỉnh nào dựa vào trainable_weights khi áp dụng update gradient).

Ví dụ: thiết lập <layer>.trainable = False

```
layer = keras.layers.Dense(3)
layer.build((None, 4)) # Create the weights
layer.trainable = False # Freeze the layer

print("weights:", len(layer.weights))
print("trainable_weights:", len(layer.trainable_weights))
print("non_trainable_weights:", len(layer.non_trainable_weights))
```

```
OUTPUT:
weights: 2
trainable_weights: 0
non_trainable_weights: 2
```

Khi một bộ trọng số (weights) bị đổi từ có thể đào tạo (trainable) trở thành không thể đào tạo (trainable = False), giá trị của nó sẽ không còn được cập nhật trong quá trình huấn luyện (training).

Trong ví dụ dưới ta xây dựng mô hình với 02 layer, trong đó một layer sẽ bị đông cứng bằng việc thiết lập trainable = False. Weights của layer bị đông cứng sẽ được lấy ngẫu nhiên. Chúng ta thực hiện huấn luyện và kiểm tra lại weights của layer.

```
import numpy as np
import keras

# Make a model with 2 layers
layer1 = keras.layers.Dense(3, activation="relu")
layer2 = keras.layers.Dense(3, activation="sigmoid")
model = keras.Sequential([keras.Input(shape=(3,)), layer1, layer2])

# Freeze the first layer
layer1.trainable = False

# Keep a copy of the weights of layer1 for later reference
initial_layer1_weights_values = layer1.get_weights()
print(initial_layer1_weights_values)

# Train the model
model.compile(optimizer="adam", loss="mse")
model.fit(np.random.random((2, 3)), np.random.random((2, 3)))

# Check that the weights of layer1 have not changed during training
final_layer1_weights_values = layer1.get_weights()
np.testing.assert_allclose(
    initial_layer1_weights_values[0], final_layer1_weights_values[0]
)
np.testing.assert_allclose(
    initial_layer1_weights_values[1], final_layer1_weights_values[1]
)

print(final_layer1_weights_values )
```

Tính đệ quy trong việc thiết lập của thuộc tính trainable

Nếu ta thiết lập trainable = False trên một mô hình hoặc bất kỳ Layer nào có Layer con (Layer bên trong), tất cả các Layer con của nó cũng sẽ không thể đào tạo được (trainable = False).

```
inner_model = keras.Sequential([
    keras.Input(shape=(3,)),
    keras.layers.Dense(3, activation="relu"),
    keras.layers.Dense(3, activation="relu"),
])

model = keras.Sequential([
    keras.Input(shape=(3,)),
    inner_model,
    keras.layers.Dense(3, activation="sigmoid"),
])

model.trainable = False # Freeze the outer model

assert inner_model.trainable == False # All layers in `model` are now frozen
assert inner_model.layers[0].trainable == False # `trainable` is propagated recursively
```

Lúc đó các layer phía trong đều có trainable = False.

Quy trình học chuyển giao (Transfer Learning)

Từ các thông tin trên, chúng ta đi đến cách thức triển khai quy trình học chuyển giao điển hình trong Keras:

1. Khởi tạo một mô hình cơ sở và tải trọng số đã được training trước (pre-trained) vào đó.
2. Đóng băng tất cả các lớp trong mô hình cơ sở bằng cách thiết lập trainable = False.
3. Tạo một mô hình mới trên đầu ra của một (hoặc nhiều) layer từ mô hình cơ sở.
4. Đào tạo mô hình mới của bạn trên tập dữ liệu mới.

Lưu ý rằng một quy trình thay thế, với số lượng weights ít hơn, có thể là:

1. Khởi tạo một mô hình cơ sở và tải trọng số đã được đào tạo trước vào đó.
2. Chạy tập dữ liệu mới của chúng ta qua đó và ghi lại đầu ra của một (hoặc nhiều) layer từ mô hình cơ sở. Điều này được gọi là **trích xuất đặc trưng**.
3. Sử dụng kết quả đầu ra đó làm dữ liệu đầu vào cho một mô hình mới nhỏ hơn.

Một lợi thế chính của quy trình làm việc thứ hai là bạn chỉ chạy mô hình cơ sở một lần trên dữ liệu của mình, thay vì một lần cho mỗi giai đoạn huấn luyện (training) lại. Vì vậy, nó nhanh hơn và rẻ hơn rất nhiều.

Tuy nhiên, một vấn đề với quy trình làm việc thứ hai là nó không cho phép bạn sửa đổi dữ liệu đầu vào của mô hình mới một cách động trong quá trình đào tạo, điều này là bắt buộc khi thực hiện tăng cường dữ liệu, ví dụ. Học chuyển giao thường được sử dụng cho các tác vụ khi tập dữ liệu mới của bạn có quá ít dữ liệu để đào tạo mô hình toàn diện từ đầu và trong các tình huống như vậy, việc tăng cường dữ liệu là rất quan trọng. Vì vậy, trong phần tiếp theo, chúng ta sẽ tập trung vào quy trình làm việc đầu tiên.

Sau đây là quy trình làm việc đầu tiên trong Keras:

Đầu tiên, hãy khởi tạo một **mô hình cơ sở** với trọng số đã được đào tạo trước (pre-trained weights - Ở đây lấy ví dụ ta dựa trên cơ sở là mô hình Xception).

```
base_model = keras.applications.Xception(
```

```
weights='imagenet', # Load weights pre-trained on ImageNet.
input_shape=(150, 150, 3),
include_top=False) # Do not include the ImageNet classifier at the top.
```

Tiếp theo: Đóng băng mô hình cơ sở

```
base_model.trainable = False
```

Tạo một mô hình mới lấy đầu ra mô hình cơ sở làm đầu vào (on-top new model).

```
inputs = keras.Input(shape=(150, 150, 3))
# We make sure that the base_model is running in inference mode here,
# by passing `training=False`. This is important for fine-tuning, as you will
# learn in a few paragraphs.
x = base_model(inputs, training=False)
# Convert features of shape `base_model.output_shape[1:]` to vectors
x = keras.layers.GlobalAveragePooling2D()(x)
# A Dense classifier with a single unit (binary classification)
outputs = keras.layers.Dense(1)(x)
model = keras.Model(inputs, outputs)
```

Huấn luyện (train) mô hình mới trên tập dữ liệu mới (giả sử tên là new_dataset)

```
model.compile(optimizer=keras.optimizers.Adam(),
              loss=keras.losses.BinaryCrossentropy(from_logits=True),
              metrics=[keras.metrics.BinaryAccuracy()])
model.fit(new_dataset, epochs=20, callbacks=..., validation_data=...)
```

Lưu ý quan trọng:

➤ compile() và trainable

Gọi compile() của một đối tượng mô hình có nghĩa là "đóng băng" hành vi của mô hình đó. Điều này ngụ ý rằng các giá trị tham số có thuộc tính trainable tại thời điểm mô hình được biên dịch phải được bảo toàn trong suốt vòng đời của mô hình đó, cho đến khi compile được gọi lại. Do đó, nếu bạn thay đổi bất kỳ giá trị tham số trainable nào, hãy đảm bảo cho mô hình gọi compile() lại để các thay đổi của được tính đến.

➤ **Layer BatchNormalization**

Nhiều mô hình CNN chứa các layers BatchNormalization. Các layers đó là một trường hợp đặc biệt theo mọi cách có thể tưởng tượng được. Sau đây là một số điều cần lưu ý (dưới đây ký hiệu layer **BatchNormalization** = **bn_layer**).

- BatchNormalization chứa 2 trọng số không thể đào tạo được cập nhật trong quá trình đào tạo. Đây là các biến theo dõi giá trị trung bình và phương sai của các đầu vào.
- Khi thiết lập **bn_layer.trainable = False**, Layer BatchNormalization sẽ chạy ở chế độ suy luận và sẽ không cập nhật số liệu thống kê trung bình và phương sai của nó. Điều này không đúng với các loại Layer khác nói chung, vì [khả năng huấn luyện weights và chế độ suy luận/huấn luyện là hai khái niệm trực giao](#) (ít khi đồng thời được xuất hiện). Nhưng cả hai đều gắn liền với nhau trong trường hợp của layer BatchNormalization.
- Khi bạn giải nén một mô hình có chứa các layers BatchNormalization để thực hiện tinh chỉnh (Fine - tuning), bạn nên giữ các layer BatchNormalization suy luận bằng cách truyền **training=False** khi gọi mô hình cơ sở. Nếu không, các bản cập nhật được áp dụng cho các trọng số không thể đào tạo sẽ đột nhiên phá hủy những gì mô hình đã học được.

VÍ DỤ END-TO-END MINH HỌA TRANSFER LEARNING & FINE-TUNNING MODEL

Trong ví dụ này chúng ta sẽ sử dụng tập dữ liệu Cats_vs_Dogs, chỉ 02 phân loại với Cat – class 0 và Dogs – class 1, dữ liệu đã cho trong các bài trước. Chúng ta sẽ sử dụng mô hình cơ sở là Xception, với trọng số đã được đào tạo trước (pre-trained) trên dữ liệu ImageNet, sau đó bổ sung các Layer mới để áp dụng cho tập dữ liệu mới với số phân lớp ít hơn. Chương trình đầy đủ các bạn tham khảo trong tập **Lab02_1_B_Morden_CNNs_Fine_Tuning**.

(i) Đọc dữ liệu

- Nếu các bạn đã tải dữ liệu về máy từ bài thực hành trước, hãy sử dụng lại công cụ [keras.utils.image_dataset_from_directory](#) – như trong bài về CNN tự xây dựng – để chuyển dữ liệu trong các thư mục về nhãn tương ứng.

- Nếu các bạn chưa có dữ liệu, hoặc thực nghiệm trên google colab, thì có thể sử dụng công cụ TFDS để lấy dữ liệu. Dưới đây là đoạn lệnh sử dụng TFDS để đọc dữ liệu, trong đó sử dụng 40% mẫu dữ liệu đầu tiên làm tập training; 10% dữ liệu tiếp theo làm Validation và 10% sau đó làm Test (tổng cộng sử dụng 60% dữ liệu)

```
tfds.disable_progress_bar()

train_ds, validation_ds, test_ds = tfds.load(
    "cats_vs_dogs",
    # Reserve 10% for validation and 10% for test
    split=["train[:40%]", "train[40%:50%]", "train[50%:60%]"],
    as_supervised=True, # Include labels
)

print(f"Number of training samples: {train_ds.cardinality()}")
print(f"Number of validation samples: {validation_ds.cardinality()}")
print(f"Number of test samples: {test_ds.cardinality()}")
```

Ta thử in ra một số mẫu hình ảnh ở phần đầu của tập training, với kích thước các hình ảnh đầu vào khác nhau và nhãn là 0 – Cats và/hoặc 1 – Dogs

```
plt.figure(figsize=(10, 10))
for i, (image, label) in enumerate(train_ds.take(9)):
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(image)
    plt.title(int(label))
    plt.axis("off")
```

Chuẩn hóa dữ liệu

Hình ảnh gốc có nhiều kích cỡ khác nhau, với 3 kênh màu (mỗi pixel bao gồm 3 giá trị thuộc [0, 255] - giá trị RGB). Chúng ta cần thực hiện 2 việc:

Chuẩn hóa thành kích thước hình ảnh cố định: Resize ảnh về 150x150.

Sử dụng một layer Normalization chuẩn hóa các giá trị pixel trong khoảng từ [-1, 1] – phần này sẽ được ghép như là một layer của mô hình. Đây chỉ là một ví dụ minh họa nên chúng ta sẽ thực hiện ít nhất có thể các bước tiền xử lý trước trước khi truy cập mô hình.

Tại đây, chúng ta sẽ thực hiện thay đổi kích thước hình ảnh trong data pipeline (vì mạng CNN sau đó chỉ có thể xử lý các batch dữ liệu liên tiếp) và chúng ta sẽ thực hiện việc thay đổi giá trị đầu vào như một phần của mô hình khi chúng ta tạo mô hình. Đoạn lệnh thay đổi kích thước hình ảnh thành 150x150:

```
resize_fn = keras.layers.Resizing(150, 150)

train_ds = train_ds.map(lambda x, y: (resize_fn(x), y))
validation_ds = validation_ds.map(lambda x, y: (resize_fn(x), y))
test_ds = test_ds.map(lambda x, y: (resize_fn(x), y))
```

Sử dụng tăng cường dữ liệu (data augmentation) ngẫu nhiên

Để tăng và làm đa dạng hóa dữ liệu, cân bằng các lớp, chúng ta sử dụng ngẫu nhiên một số phép biến hình (xoay ảnh, biến dạng, tăng giảm sáng – phần xử lý ảnh) để tạo mẫu mới trên các ảnh đang có:

```
augmentation_layers = [
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
]

def data_augmentation(x):
    for layer in augmentation_layers:
        x = layer(x)
    return x

train_ds = train_ds.map(lambda x, y: (data_augmentation(x), y))
```

Phân lô dữ liệu (batch) và sử dụng một số công cụ nhằm tăng tốc độ tải các lô dữ liệu khi xử lý:

```
from tensorflow import data as tf_data

batch_size = 64

train_ds = train_ds.batch(batch_size).prefetch(tf_data.AUTOTUNE).cache()
validation_ds = validation_ds.batch(batch_size).prefetch(tf_data.AUTOTUNE).cache()
test_ds = test_ds.batch(batch_size).prefetch(tf_data.AUTOTUNE).cache()
```

Trong đoạn code này chúng ta có thể thay đổi kích thước Batch (batch_size) phù hợp với cấu hình hệ thống.

Hiển thị một số ảnh trong batch đầu tiên, sau khi đã qua các biến đổi tăng cường dữ liệu:

```
for images, labels in train_ds.take(1):
    plt.figure(figsize=(10, 10))
    first_image = images[0]
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        augmented_image = data_augmentation(np.expand_dims(first_image, 0))
        plt.imshow(np.array(augmented_image[0]).astype("int32"))
        plt.title(int(labels[0]))
        plt.axis("off")
```

Xây dựng một mô hình

Bây giờ chúng ta hãy xây dựng một mô hình dựa trên mô hình cơ sở là Xception theo bản thiết kế mà chúng ta đã phân tích trước đó. Lưu ý:

- Bổ sung một layer Rescaling để điều chỉnh giá trị đầu vào (từ [0, 255] về [-1, 1]).
- Bổ sung một layer Dropout trước lớp phân loại để tránh overfit.
- Đặt tham số **training=False** khi gọi mô hình cơ sở để mô hình chạy ở chế độ suy luận, do đó số liệu thống kê batchnorm không được cập nhật ngay cả sau khi ta mở khóa mô hình cơ sở để tiến hành tinh chỉnh (fine-tuning).

```
base_model = keras.applications.Xception(
    weights="imagenet", # Load weights pre-trained on ImageNet.
    input_shape=(150, 150, 3),
    include_top=False,
) # Do not include the ImageNet classifier at the top.

# Freeze the base_model
base_model.trainable = False

# Create new model on top
inputs = keras.Input(shape=(150, 150, 3))

# Pre-trained Xception weights requires that input be scaled
# from (0, 255) to a range of (-1., +1.), the rescaling layer
# outputs: `(inputs * scale) + offset`
scale_layer = keras.layers.Rescaling(scale=1 / 127.5, offset=-1)
x = scale_layer(inputs)

# The base model contains batchnorm layers. We want to keep them in inference mode
# when we unfreeze the base model for fine-tuning, so we make sure that the
# base_model is running in inference mode here.
x = base_model(x, training=False)
x = keras.layers.GlobalAveragePooling2D()(x)
x = keras.layers.Dropout(0.2)(x) # Regularize with dropout
outputs = keras.layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

model.summary(show_trainable=True)
```

Training layer TOP (Layer bổ sung)

Trước hết chúng ta thực đóng băng toàn bộ phần CNN (Xception – Mô hình cơ sở) và chỉ thực hiện training lại lớp FC trên cùng. Toàn bộ trọng số của phần CNN backbone chúng ta chuyển giao từ pre-trained.

```

model.compile(
    optimizer=keras.optimizers.Adam(),
    loss=keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=[keras.metrics.BinaryAccuracy()],
)

epochs = 2
print("Fitting the top layer of the model")
model.fit(train_ds, epochs=epochs, validation_data=validation_ds)

```

Thực hiện fine-tuning (1 epoch) trên toàn bộ mô hình

Trong phần cuối, ta mở khóa (bỏ đóng băng mô hình cơ sở) mô hình cơ sở và huấn luyện toàn bộ mô hình từ đầu đến cuối với learning rate chọn đủ nhỏ.

Ở đây có một điểm quan trọng cần chú ý: Mặc dù mô hình cơ sở có thể được huấn luyện lại, nhưng nó vẫn đang chạy ở chế độ suy luận vì chúng ta đã thiết lập chế độ **training=False** trong lời gọi đối tượng mô hình này (Xception cơ sở) khi chúng ta xây dựng mô hình. Điều này có nghĩa là các lớp chuẩn hóa lô (Batch Normalization) bên trong sẽ không cập nhật tham số thống kê (mean, variance) trong lô của chúng. Nếu chúng bị thay đổi, chúng sẽ phá hỏng các biểu diễn mà mô hình đã học được cho đến bước này.

```

# Unfreeze the base_model. Note that it keeps running in inference mode
# since we passed `training=False` when calling it. This means that
# the batchnorm layers will not update their batch statistics.
# This prevents the batchnorm layers from undoing all the training
# we've done so far.
base_model.trainable = True
model.summary(show_trainable=True)

model.compile(
    optimizer=keras.optimizers.Adam(1e-5), # Low learning rate
    loss=keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=[keras.metrics.BinaryAccuracy()],
)

epochs = 1
print("Fitting the end-to-end model")
model.fit(train_ds, epochs=epochs, validation_data=validation_ds)

```

Hãy thực hiện lệnh, quan sát kiến trúc của Model và sau đó thực hiện với số epoch = 10 (BÀI THỰC HÀNH Ở NHÀ).

Cuối cùng, hãy thực hiện dự đoán trên tập dữ liệu TEST:

```

print("Test dataset evaluation")
model.evaluate(test_ds)

```

Bài tập thực hành:

Tổng hợp các bước để có code hoàn chỉnh và chạy thực nghiệm.

B. PRETRAINED MODERN CNN & FINE – TUNING MODEL (TORCH BASED)

Trong phần này, chúng ta sẽ dựa vào lý thuyết ở phần trên để đọc hiểu và thực hiện thao tác thay đổi các tham số để tiến hành thực nghiệm cũng như đánh giá kết quả trên tập dữ liệu Chó – Mèo – Pandas đã có.

Phần đọc dữ liệu, chia train : test (hoặc Validation) cũng như gán nhãn theo tên thư mục chúng ta có thể tham khảo lại Lab02.1B.

Chương trình tham khảo có trong tệp đính kèm: **pytorch_transfer_finetune.ipynb**. Cấu trúc của các đoạn chương trình có thông tin như sau (các mục gần như khớp với các đoạn lệnh trong notebook)

1) Cấu hình chung (Configuration)

Các biến bạn nên chỉnh trước khi chạy:

DATA_ROOT: Path("data_root") = Thư mục dữ liệu gốc.

Chuẩn bị dữ liệu theo một trong hai cách

Cách tổ chức 1 (tự chia train/val):

- DATA_ROOT/
- class_a/ img1.jpg ...
- class_b/ img2.jpg ...

Cách tổ chức 2 (đã chia sẵn):

DATA_ROOT/
train/class_a/...
train/class_b/...
val/class_a/...
val/class_b/...

Nếu theo cách 2, đặt **NEED_RANDOM_SPLIT=False**.

- **NEED_RANDOM_SPLIT:** True|False
 - True: Notebook tự chia train/val theo TRAIN_RATIO.
 - False: Dùng đúng train/ và val/ đã có sẵn.
- **TRAIN_RATIO:** 0.8
Tỉ lệ train khi tự chia. Dùng 0.7–0.9 là phổ biến.
- **RANDOM_SEED:** 42
SEED để chia dữ liệu tái lập kết quả. Thay đổi để tạo split khác.
- **MODEL_NAME:** "resnet18"
Tên mô hình pretrained. Tùy chọn đã cài sẵn trong notebook:
 - resnet18 (nhẹ, nhanh, baseline tốt)
 - resnet50 (mạnh hơn, nặng hơn)
 - efficientnet_b0 (nhẹ, hiệu quả)
 - mobilenet_v3_small (rất nhẹ, hợp thiết bị yếu)
- **IMAGE_SIZE:** 224
Kích thước ảnh đầu vào. 224 là chuẩn cho nhiều backbone; có thể 256/299 tùy model (nhưng nhớ đổi transform tương ứng). Kích thước chuẩn của đầu vào các bạn có thể tự tra cứu.
- **BATCH_SIZE: 32, NUM_EPOCHS: 10**
 - BATCH_SIZE phụ thuộc RAM GPU/CPU. Nếu "out of memory", giảm 32 → 16 → 8.

- `NUM_EPOCHS`: số vòng lặp huấn luyện. Dữ liệu nhỏ có thể 10–20; dữ liệu lớn 30–100.
- **`LR_BACKBONE = 1e-4, LR_CLASSIFIER = 1e-3`**
Hai tốc độ học (learning rate) khác nhau:
 - Backbone (phần trích đặc trưng pretrained) nên LR nhỏ hơn để tránh “quên” tri thức cũ.
 - Classifier (layer cuối thay mới) nên LR lớn hơn để học nhanh.
Bạn có thể thử tỉ lệ 1:5 đến 1:10 giữa backbone:head.
- **`FREEZE_STRATEGY: "partial"`**
Chiến lược “đóng băng”:
 - **`"all_frozen"`**: Chỉ học classifier (feature extractor cố định). Nhanh, ít overfit, phù hợp dữ liệu nhỏ.
 - **`"partial"`**: Mở khóa (unfreeze) một vài block cuối + head. Cân bằng tốc độ/chất lượng.
 - **`"none"`**: Mở toàn bộ (full fine-tune). Chậm hơn, yêu cầu dữ liệu lớn hơn, có thể đạt tốt nhất nếu dữ liệu đủ.
- **`UNFREEZE_LAST_BLOCKS: 1`**
Khi `"partial"`, số block cuối được mở. Với ResNet, block là layer4, layer3,... Tăng 1→2 nếu muốn mở nhiều hơn.
- **`USE_AMP: True`**
Bật mixed precision (float16/32) để tăng tốc và tiết kiệm VRAM trên GPU hỗ trợ.
- **`NUM_WORKERS: 2`**
Số worker đọc dữ liệu. Trên Linux/GPU có thể tăng 4–8. Trên Windows đôi khi 0–2 ổn định hơn.
- **`OUTPUT_DIR, CHECKPOINT_PATH`**
Thư mục/lưu checkpoint `"best_model.pt"`. Có thể đổi tên cho mỗi thí nghiệm.

Kỳ vọng:

Thiết lập hợp lý giúp huấn luyện ổn định, nhanh hơn và dễ so sánh thí nghiệm (chỉ cần đổi vài tham số).

2) Nhập thư viện & thiết bị

- Kiểm tra device = `"cuda"` nếu có GPU, ngược lại `"cpu"`.
- Thử import `sklearn.metrics` để dùng `classification_report` và `confusion_matrix`. Nếu không có, notebook có sẵn phiên bản tính tay.

Kỳ vọng:

In ra `"Using device: cuda/cpu"`. Nếu dùng CPU, huấn luyện sẽ lâu hơn.

3) Biến đổi dữ liệu (Transforms)

- `train_transform`:
 - `RandomResizedCrop(IMAGE_SIZE), RandomHorizontalFlip()` để tăng cường dữ liệu (augmentation).
 - Chuẩn hóa theo ImageNet mean/std (bắt buộc nếu dùng pretrained từ ImageNet).
- `val_transform`:
 - `Resize(IMAGE_SIZE + 32), CenterCrop(IMAGE_SIZE),` rồi `Normalize`.

Chỉnh thế nào?

- Dữ liệu ít/overfit: thêm `ColorJitter, RandomRotation, RandomErasing`.
- Dữ liệu đã rất đa dạng: giảm bớt augmentation để tránh nhiễu.

Kỳ vọng:

Tổng quát hóa tốt hơn, accuracy val không quá lệch so với train.

4) Datasets & Chia tập

Hàm `build_datasets(...)`:

- Nếu có train/ & val/ và **NEED_RANDOM_SPLIT=False**: dùng trực tiếp.
- Nếu chỉ có một root với subfolder class: dùng ImageFolder đọc tất cả, rồi random_split theo TRAIN_RATIO, đảm bảo cố định bằng RANDOM_SEED.
- Áp transform theo đúng split (augment mạnh cho train, nhẹ cho val).

Sau đó tạo DataLoader cho train/val:

- shuffle=True cho train, False cho val.
- pin_memory=True giúp tăng tốc (đặc biệt với GPU).

Kỳ vọng:

In ra danh sách Classes: ['class_a', 'class_b', ...]. Nếu sai tên lớp → kiểm tra tên thư mục con.

5) Xây mô hình (Model factory)

Hàm build_model(MODEL_NAME, num_classes):

- Tải backbone pretrained (weights=..._Weights.DEFAULT).
- Thay head phân loại cuối bằng nn.Linear(in_features, num_classes).
- Trả thêm danh sách “tên module” để phân nhóm tham số (backbone vs head).

Chỉnh thế nào?

- Thay MODEL_NAME.
- Muốn đổi kiến trúc head phức tạp hơn (dropout, MLP 2 tầng), bạn có thể sửa phần gán m.fc hoặc m.classifier[...].

Kỳ vọng:

Model khởi tạo với trọng số ImageNet, head mới random init. In ra số tham số trainable/total.

6) Chiến lược đóng băng (Freeze strategy)

Hàm set_freeze_strategy(...):

- Mặc định **đóng băng tất cả**: param.requires_grad=False.
- Tùy **FREEZE_STRATEGY**:
 - all_frozen: chỉ head mở (trainable).
 - partial: head mở + “N block cuối” mở (ResNet: layer4/layer3...; MobileNet/EfficientNet: phần cuối features).
 - none: mở hết.

Chỉnh thế nào?

- Tăng **UNFREEZE_LAST_BLOCKS** để fine-tune sâu hơn (nhưng chậm hơn, dễ overfit nếu data nhỏ).

Kỳ vọng:

In: “Trainable params: X / Y”. Nếu bạn chọn all_frozen, số trainable nhỏ (chỉ vài trăm nghìn). none: gần Y.

7) Tối ưu hóa (Optimizer, Loss, Scheduler)

- Hàm param_groups(...) gom tham số backbone & head thành 2 nhóm, áp 2 LR khác nhau.
- optimizer = AdamW(..., weight_decay=1e-4)
 - AdamW là lựa chọn tốt, bền.
 - weight_decay=1e-4 giúp regularization, có thể 1e-5–1e-4.
- criterion = CrossEntropyLoss() cho phân loại đa lớp.
- scheduler = CosineAnnealingLR(T_max=NUM_EPOCHS-1) làm LR giảm dần theo “cosine”.

Chỉnh thế nào?

- Thử SGD(momentum=0.9, nesterov=True) nếu muốn kỷ luật LR và tổng quát hóa tốt.
- Thử OneCycleLR khi biết tổng batch/epoch cố định.

Kỳ vọng:

Loss giảm dần; accuracy tăng dần. Nếu không: LR quá cao/ thấp, data leak, hoặc label sai.

8) Vòng lặp huấn luyện & đánh giá

- **USE_AMP** bật GradScaler + autocast để huấn luyện FP16 an toàn trên GPU.
- **train_one_epoch(...)**:
 - Chế độ model.train(); tính loss, backward, update.
 - Ghi nhận running_loss và running_correct.
- **evaluate(...)**:
 - torch.no_grad() + model.eval(); trả val_loss, val_acc, và y_true, y_pred cho metrics sau.

Checkpoint tốt nhất:

- Lưu khi val_acc tốt hơn trước (torch.save state_dict về CPU để portable).
- File: OUTPUT_DIR/best_model.pt chứa: model_state, class_names, model_name.

Kỳ vọng:

Log mỗi epoch dạng:

Epoch 003/010 | train_loss=... acc=... | val_loss=... acc=... | time=...s

- Nếu val_acc không tăng, thử:
 - Tăng số epoch.
 - Giảm LR_BACKBONE và/hoặc LR_CLASSIFIER.
 - Đổi freeze strategy (sang partial hoặc none).
 - Thêm/bớt augmentation.
-

9) Đồ thị học (Learning Curves)

- Vẽ loss (train/val) & accuracy (train/val) theo epoch.
 - **Giải thích diễn giải:**
 - train_loss ↓ nhưng val_loss ↑: overfit. Tăng regularization, augmentation, hoặc tăng data.
 - cả 2 loss không giảm: LR sai, bug data/label, hoặc mô hình quá nhỏ/không đủ epoch.
-

10) Chỉ số đánh giá & Confusion Matrix

- Nếu có scikit-learn:
 - classification_report: precision/recall/F1 **per class** + macro avg.
 - confusion_matrix: ma trận nhầm lẫn.
- Nếu không có, notebook tính thủ công P/R/F1.
- Vẽ heatmap confusion matrix bằng matplotlib.

Chỉnh thế nào?

- Bạn có thể sắp xếp label theo tần suất, hoặc chuẩn hóa theo hàng/cột khi vẽ.

Kỳ vọng:

- Precision/Recall/F1 macro hợp lý ($>0.7-0.9$ với bộ dữ liệu sạch & đủ).
 - Confusion matrix đậm (hoặc sáng nhất trên) diagonal chính (dự báo đúng nhiều).
 - Xem ô off-diagonal lớn để biết lớp hay bị nhầm với ai $\rightarrow$ gợi ý tăng dữ liệu/augment hoặc chia lại lớp.
-

11) Dự đoán/Suy luận (Inference) & Tải checkpoint

- `load_checkpoint(path)`: dựng lại model đúng `MODEL_NAME`, nạp `state_dict`, trả về `model.eval()` + `class_names`.
- `predict_image(img_path, model, class_names)`: đọc ảnh, transform như val, trả (label, confidence).

Kỳ vọng:

- Dùng cho test nhanh:
 - `model, classes = load_checkpoint(CHECKPOINT_PATH)`
 - `predict_image("demo.jpg", model, classes)`
 - Nếu confidence thấp (ví dụ $0.3-0.5$) $\rightarrow$ ảnh khó, hoặc model cần thêm data/epoch.
-

12) Mẹo thực tiễn & mở rộng

Khi nào “đóng băng toàn bộ” (`all_frozen`)?

- Dữ liệu rất ít (vài chục–vài trăm ảnh), muốn kết quả nhanh/chắc, tránh overfit.

Khi nào “partial”?

- Dữ liệu vừa phải (vài nghìn ảnh). Đây là mặc định “an toàn” cho đa số.

Khi nào “none”?

- Dữ liệu lớn/đa dạng; cần chất lượng cao nhất; có GPU mạnh và thời gian huấn luyện.

Tối ưu thêm:

- Early stopping (dừng khi `val_loss` không cải thiện 5–10 epoch).
- Gradient clipping (`torch.nn.utils.clip_grad_norm_`) nếu loss dao động mạnh.
- TensorBoard logging (phân tích LR, histogram, images).
- AutoAugment/RandAugment/CutMix/MixUp nếu dữ liệu đa dạng.
- Tối ưu LR: thử OneCycleLR, hoặc ReduceLROnPlateau nếu val không cải thiện.
- Export ONNX/TorchScript để triển khai.
- Grad-CAM để giải thích vùng model chú ý.

Xử lý lỗi thường gặp:

- “Found 0 files”: kiểm tra DATA_ROOT và phần đuôi ảnh .jpg|.png.
 - Lệch số lớp (dim mismatch): đổi DATA_ROOT → số lớp thay đổi; xóa checkpoint cũ hoặc train lại head tương ứng.
 - OOM (hết VRAM): giảm BATCH_SIZE, bật USE_AMP=True, giảm IMAGE_SIZE.
-

13) Kết quả mong đợi (tổng quan)

- Với bộ dữ liệu cân bằng, đủ ảnh (≥ 100 ảnh/lớp), resnet18 + partial + 10–20 epoch thường đạt **acc 85–95%** trên các tác vụ phổ thông.
- Bộ dữ liệu khó/không cân bằng: hãy xem classification_report theo lớp để biết lớp yếu (recall thấp), từ đó:
 - Bổ sung dữ liệu lớp yếu.
 - Dùng WeightedRandomSampler hoặc class_weight cho CrossEntropyLoss.
 - Điều chỉnh augmentation theo đặc thù lỗi nhầm.

BÀI TẬP: Hãy chọn một mô hình hiện đại, sau đó thử huấn luyện/dự đoán trên tập ảnh chó-mèo-pada đã có với các tham số tinh chỉnh khác nhau, lập bảng so sánh kết quả và đưa ra đánh giá, giải thích nguyên nhân.